Industrial Internship Report on

DRAFTLINE: BLOG CONTENT MANAGEMENT SYSTEM

Prepared by

JEET SHAW

Executive Summary

This report documents the Full Stack Web Development internship project developed against the Content Management System for a Blog brief supplied for the UCT internship programme.

Draftline is a full-stack editorial CMS that combines a public journal with a protected administration workspace. It supports rich-text blog writing, draft and publish states, editable visual pages with reorderable blocks, image management, SEO metadata, and database-backed publishing.

The implementation uses Next.js App Router, React, TypeScript, Tailwind CSS, Prisma ORM with PostgreSQL, Tiptap for rich-text editing, dnd-kit for block reordering, and Cloudinary for image assets. Authentication and validation are implemented in the application API layer.

Source code: GitHub repository | Live site: Draftline on Vercel

TABLE OF CONTENTS

1 Preface....3
2 Introduction....4
2.1 About UniConverge Technologies Pvt Ltd....4
2.2 About upskill Campus....8
2.3 The IoT Academy....9
2.4 Objectives of this Internship program....9
2.5 References....10
2.6 Glossary....10
3 Problem Statement....11
4 Existing and Proposed solution....12
5 Proposed Design/ Model....13
5.1 High Level Diagram (if applicable)....13
5.2 Low Level Diagram (if applicable)....13
5.3 Interfaces (if applicable)....13
6 Performance Test....14
6.1 Test Plan/ Test Cases....14
6.2 Test Procedure....14
6.3 Performance Outcome....14
7 My learnings....15
8 Future work scope....16

1 Preface

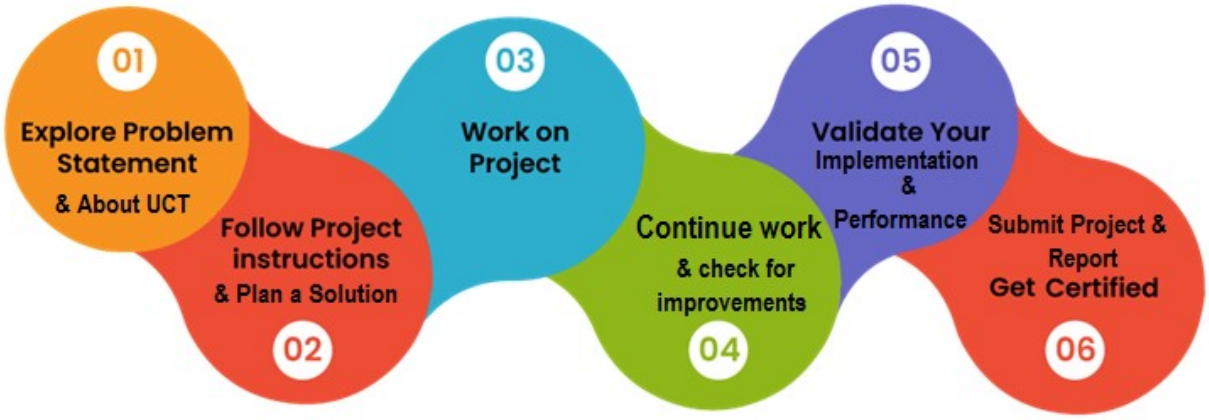
This report summarises the planning, implementation, deployment preparation, and documentation of Draftline, a Content Management System for a Blog created for the Full Stack Web Development internship brief.

A practical internship connects classroom concepts with a complete software lifecycle: understanding a requirement, designing a solution, building user-facing features, persisting data, managing security, testing workflows, and documenting the outcome.

Draftline addresses the effort involved in maintaining a blog by bringing post authoring, draft/publish control, reusable page layouts, media management, and public publishing into one web application.

The internship programme facilitated by upskill Campus and The IoT Academy with UCT provided the project brief and a framework for applying full-stack development concepts to an end-to-end product.

The work was organised iteratively: analyse the brief, plan the data model and routes, implement public and admin interfaces, connect APIs and the database, add authentication and media handling, deploy the site, and prepare documentation.



The project strengthened my understanding of frontend-backend integration, relational data modelling, authentication, input validation, content lifecycle design, responsive interfaces, and deployment considerations.

I am thankful to the upskill Campus, The IoT Academy, and UCT teams and instructors for the internship opportunity and learning material. I also acknowledge the documentation and open-source tools that supported the implementation.

My message to juniors and peers is to choose a clearly defined problem, build a working version early, test every critical workflow, document known limitations honestly, and keep the code organised in version control.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



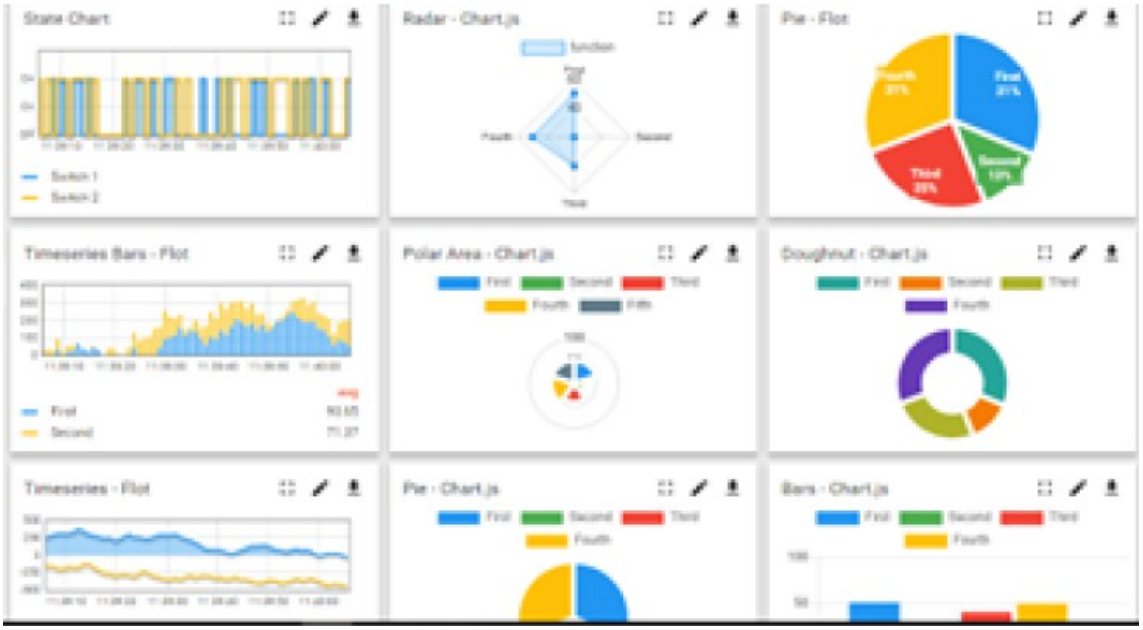
i. UCT IoT Platform (uct Insight)

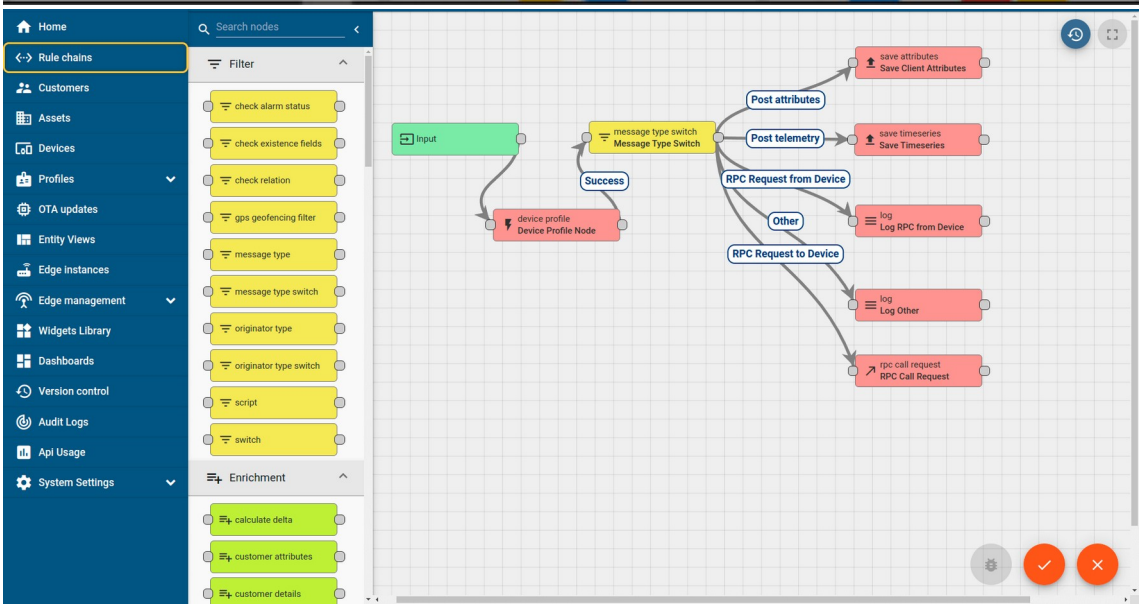
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

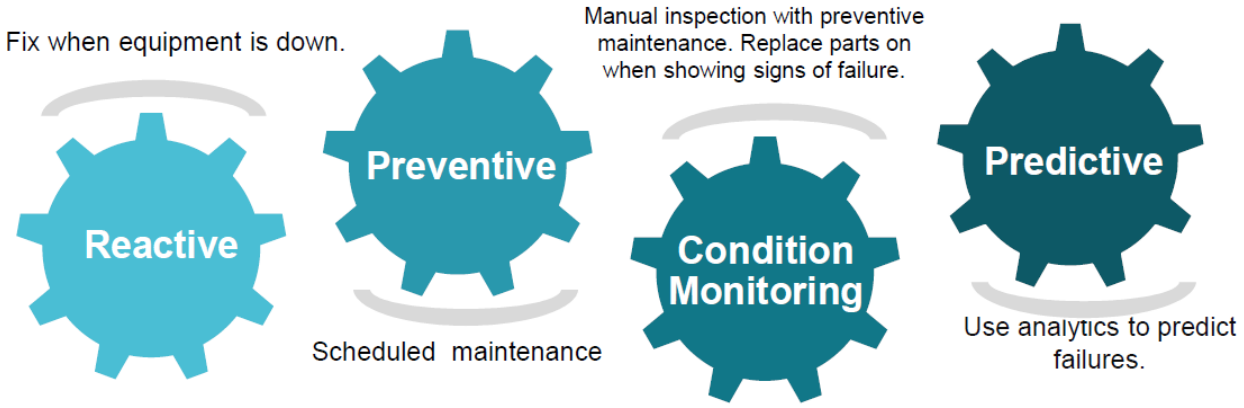


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

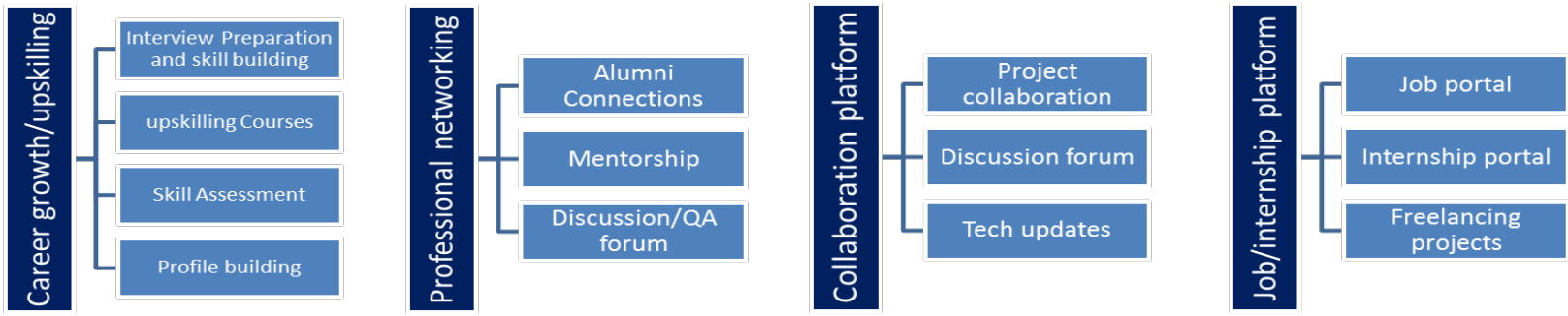
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.


Career growth/upskilling
Interview Preparation and skill building
upskilling Courses
Skill Assessment
Profile building
Professional networking
Alumni Connections
Mentorship
Discussion/QA forum
Collaboration platform
Project collaboration
Discussion forum
Tech updates
Job/internship platform
Job portal
Internship portal
Freelancing projects

2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- ☛ get practical experience of working in the industry.
- ☛ to solve real world problems.
- ☛ to have improved job prospects.
- ☛ to have Improved understanding of our field and its applications.
- ☛ to have Personal growth like better communication and problem solving.

2.5 References

[1] UCT Full Stack Internship project brief: “Content Management System for a Blog.”
[2] Draftline source repository and project documentation: https://github.com/jeetshawXX/draftline
[3] Next.js documentation: https://nextjs.org/docs | Prisma documentation: https://www.prisma.io/docs

2.6 Glossary

Terms	Acronym
CMS	Content Management System
API	Application Programming Interface
ORM	Object-Relational Mapping
SEO	Search Engine Optimization
HTTPS	HTTP Secure (encrypted web transport)

3 Problem Statement

The assigned problem statement was to develop a full-stack blog CMS similar in core purpose to platforms such as WordPress and Drupal. The system should let a website owner manage content, design page layouts with reusable sections, and publish blog posts that are stored in a database and served through public web routes.

Problem addressed by Draftline: routine changes to blog content and page layouts should not require direct code edits. Writers need a structured editor, site owners need a visual page builder and media support, and visitors should see only content that has been published.

Project objectives:

• Provide public journal, article, and custom-page routes backed by persistent content.

• Provide a protected admin studio for posts, pages, images, and publishing status.

4 Existing and Proposed solution

Existing solutions: WordPress and Drupal provide mature CMS workflows, rich publishing ecosystems, and many extensions. For a focused internship implementation, adopting a large existing platform would provide less opportunity to demonstrate the internal frontend, API, data-model, authentication, and rendering workflows requested in the assignment.

Proposed solution: Draftline implements a focused blog CMS in a single Next.js application. It combines a public journal with an admin studio, a Tiptap rich-text editor, saved draft/published states, a drag-and-drop page builder, PostgreSQL persistence through Prisma, and a media library backed by Cloudinary.

Value addition: reusable Hero, Text, Image, Feature Grid, and Call to Action blocks; slug-based public pages; SEO fields; validation and sanitization of content; protected mutation APIs; responsive layouts; and a documented manual acceptance-test plan.

Live deployment: https://draftlinejeet.vercel.app/

4.1 Code submission (GitHub repository)

Source code: https://github.com/jeetshawXX/draftline

4.2 Project report (GitHub link)

Project documentation: docs/PROJECT_REPORT.md

5 Proposed Design/ Model

Draftline uses a layered full-stack architecture. The browser serves public pages and the admin studio; Next.js routes render pages and expose API handlers; Prisma accesses PostgreSQL; Cloudinary stores image assets. Authentication, validation, and content sanitization are enforced in the server-side layer.

5.1 High-Level Architecture

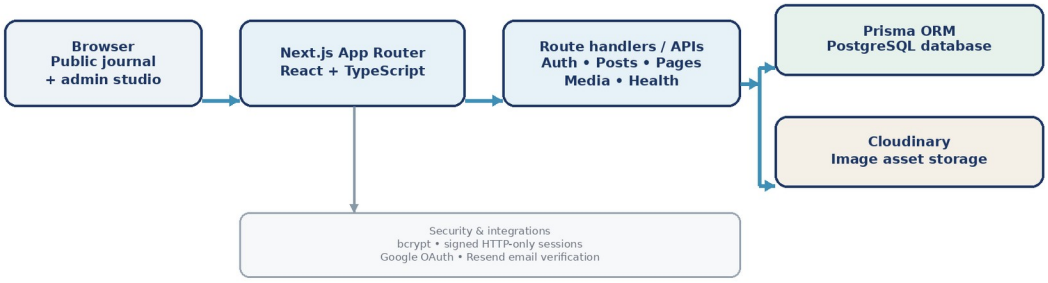


Figure 1: High-level architecture of Draftline

5.2 Low-Level Design and Data Flow

Data flow: the Next.js UI calls route handlers for authentication, posts, pages, media, overview data, and health checks; validated requests are persisted through Prisma to PostgreSQL.

Core models are User, Post, Page, Media, Setting, and EmailVerificationCode. Public routes return PUBLISHED content only; DRAFT records stay in the admin workflow until published.

5.3 Interfaces

Public interfaces: journal homepage (/), post details (/blog/[slug]), and custom pages (/p/[slug]). Admin interfaces: login, dashboard, post editor, visual page builder, and media library.

Tiptap supports rich text; dnd-kit reorders page blocks with keyboard movement alternatives. Protected create/update/delete APIs require an administrator session; the media API validates images and stores them in Cloudinary.

6 Performance Test

The main constraints considered were content visibility correctness, database consistency, authentication boundaries, image upload validation, responsive rendering, and deployment persistence. The source repository contains a manual test plan for these workflows; however, the committed test-execution record is marked “Not run,” so no unrecorded pass results or performance figures are claimed.

Functional testing should be performed on the deployed site and, where necessary, in a locally seeded environment. Tests must record date, browser/environment, commit, expected behaviour, observed result, and evidence for any failures.

No measured response-time, throughput, concurrent-user, or load-test figures were supplied. These are left as future validation work rather than estimated or invented results.

6.1 Test Plan / Test Cases

TC-01 — Public publishing: verify published posts/pages render, while drafts remain unavailable on public routes. (Execution record: pending.)

TC-02 — Authentication: verify valid login, invalid password handling, logout, and rejection of unauthenticated admin API mutations. (Pending.)

TC-03 — Content CRUD: create, edit, publish, unpublish, and delete a test post; confirm saved values persist after reload. (Pending.)

TC-04 — Page builder: add Hero/Text/Image/Feature Grid/CTA blocks, reorder them, save, reload, and verify order on the public route. (Pending.)

TC-05 — Media safety: upload allowed image types, reject unsupported/oversized files, and confirm alternative text persists. (Pending.)

TC-06 — Release checks: run typecheck, lint, production build, database health, and HTTP-to-HTTPS checks; record outputs before marking pass. (Pending.)

6.2 Test Procedure

• Configure environment variables and a seeded administrator; deploy/use the intended environment; run each case with fresh test content and a non-production test image.

• Record Pass/Fail, date, browser/environment, relevant response codes, and screenshot/log evidence in docs/TEST_PLAN.md. Repeat failed cases after fixes.

6.3 Performance Outcome

Implementation outcome: the repository contains the end-to-end page, post, authentication, database, and media workflows described in this report, and the author has supplied a live Vercel URL. Formal functional-test results, automated check outputs, and numerical performance benchmarks remain to be recorded; deployment availability alone is not presented as a performance measurement.

7 My learnings

Building Draftline helped me understand how the parts of a full-stack application work together. The most important learnings were:

• Frontend engineering: creating responsive pages, reusable React components, editor forms, admin navigation, and predictable loading/error states.

• Backend and data: designing API handlers, validating request payloads, defining Prisma models/relations, and maintaining content status and slug uniqueness.

• Authentication and safety: hashing passwords, signing expiring sessions, enforcing admin checks, validating uploads, and sanitizing rich text before rendering.

• Content tools: integrating Tiptap for structured writing, dnd-kit for reorderable page blocks, and Cloudinary for image asset storage.

• Delivery discipline: configuring environment variables, managing secrets outside source control, using GitHub, documenting setup and test cases, and deploying a live version.

8 Future work scope

Draftline meets the core educational scope of a blog CMS. The following enhancements would improve maintainability, safety, and readiness for broader use:

- Add role separation for author, editor, and administrator, and restrict public account registration through invitation or approval. The current signup path can grant admin-level access after verification and should be tightened before broad public use.
- Add automated unit/API/end-to-end tests and a CI workflow that runs type checking, linting, builds, and key publishing/authentication tests on each change.
- Add post version history, autosave, scheduled publishing, preview links for drafts, and editorial review/approval workflows.
- Add pagination, full-text search, normalized categories/tags, RSS feeds, sitemap generation, and richer analytics.
- Add production observability, structured logs, backup/restore drills, centralized rate limiting, security headers/CSP, and a formal threat review.
- Review scaling and media delivery as usage grows; keep PostgreSQL managed, improve cache strategy, and monitor Cloudinary and hosting limits.